

TourTracker – Development Protocol

Technical steps, key decisions, failures and the solutions we chose. Based on our git history and pull requests (Feb – Jul 2026).

1. Tech stack

Layer	Technology
Frontend	Angular (standalone components, MVVM pattern)
Backend	Java + Spring Boot (layered: Controller → Service → Repository)
Database	PostgreSQL via JPA/Hibernate, migrations with Flyway
Maps / routing	Leaflet (map) + OpenRouteService API (distance, time, route)
Auth	JWT tokens
Build/Run	Maven, docker-compose, GitHub Actions (CI)

2. How the project developed (chronological)

Phase 1 – Frontend first (Feb – Apr). We started with the Angular frontend and the UML diagrams. First we built routing and the main pages (home, login, register, tour details, settings) and managed tours in a local array so we could design the UI before the backend existed. Tour and tour-log CRUD, a star-rating system, and image selection were added here.

Phase 2 – MVVM refactor (Apr). The first version mixed logic and view too much. We refactored to the **MVVM pattern** (ViewModels + a storage service for persistence) and cleaned up the HTML bindings. We also built our **reusable toast component** to replace plain error messages.

Phase 3 – Backend & database (Apr 30 – May). We split the repo into **frontend/** and **backend/** folders and built the Spring Boot backend with a **layered architecture**. We set up PostgreSQL, Flyway migrations (**V1__init.sql**), JWT authentication, and moved all configuration (DB connection, JWT secret) into environment variables so no secrets are in the source. **docker-compose** was added to start everything easily.

Phase 4 – Maps, routing, images (May). We integrated **OpenRouteService + Leaflet**: distance, time and the route geometry come from ORS, the map is drawn with Leaflet. Transport types were mapped to ORS routing profiles. Server-side validation and **filesystem image storage** were added.

Phase 5 – Features & polish (Jun).

- Geolocation (user's own position on the map) as our extra feature (PR #63).
- **TourMetricsCalculator** for the **computed attributes** popularity and child-friendliness (PR #65); the full-text search was extended to also match these computed values.
- Sortable full-text search (PR #68).
- Custom exception handling that wraps data-access errors in business-layer exceptions (PR #69).
- CI pipeline with GitHub Actions and a unit-test suite (PR #71).
- Import/export of tour data (PR #73).

3. Key design decisions

- **PostgreSQL instead of H2.** We first had an H2 config but removed it and committed to PostgreSQL, since it is the required database and behaves the same in dev and "production".
- **Layered architecture + DTOs.** Controllers only talk to services, services to repositories. Entities are never returned directly; we map them to DTOs. This keeps the layers separate and matches the assignment.
- **Config outside the source.** DB credentials and the JWT secret are read from a `.env` file (`.env.example` is committed as a template), so nothing secret is in git.
- **Cache the route geometry in the database.** See failures below.
- **AOP for exception translation (design pattern).** A `DataAccessExceptionAspect` (`@Around` advice) wraps every service method and turns raw `DataAccessExceptions` into clean layer exceptions, so the frontend never sees a database error. This is documented separately as our design pattern.

4. Failures and the solutions we chose

- **Edit vs. open collided.** Clicking a tour card and clicking its edit button did the same thing. **Solution:** we add an `isEditing` flag to the URL so the two actions are distinguishable, and the submit button label changes between "create" and "edit".
- **ORS was called on every load.** Opening a saved tour re-called the OpenRouteService API just to draw the map, which is slow and wastes API quota. **Solution:** we cache the route geometry in the database when the tour is saved and reuse it, so no ORS call is needed for existing tours.
- **Child-friendliness bug.** The computed child-friendliness value was wrong at first; we fixed the calculation and later added a unit test that clamps the popularity lower bound so it can't happen again.
- **We over-engineered the security.** On one branch we added a lot of extra security (IP rate limiting, refresh-token rotation, HttpOnly cookies, a server-side token blacklist, account lockout). It was **out of scope** and made the app harder to run. **Solution:** we reverted those features and kept only the sensible parts — JWT hardening (fail fast on a missing/weak secret) and input validation (PR #64).
- **Login errors were unhelpful.** Every failure showed "invalid credentials". **Solution:** the frontend now extracts the real backend message, and the backend emits clean validation messages without internal field-name prefixes. The password hint now lists all requirements.
- **Spring Boot 4 / Jackson 3.** A change removed a manually-defined `ObjectMapper` bean, which broke the application context on startup. **Solution:** we restored the bean for Jackson 3 compatibility.
- **CI kept failing after merges.** Merging the import/export branch left a test calling a service constructor with the wrong number of parameters (a new validator argument). **Solution:** we updated the affected tests to inject the real validator and got the pipeline green again.

5. Process notes

- We worked with **feature branches and pull requests** and did code reviews on each; several commits are "address review feedback".
- **Git history is part of the documentation** — the commit messages describe most decisions above.
- The frontend was formatted with Prettier and the backend follows the standard Spring layout.

6. Result

A working two-tier Angular + Spring Boot application that fulfills the requirements: user registration/login, tour and tour-log CRUD, ORS + Leaflet routing, full-text search over stored **and** computed values, computed

popularity/child-friendliness, import/export, filesystem image storage, a reusable UI component, a design pattern, logging, and a unit-test suite running in CI.